

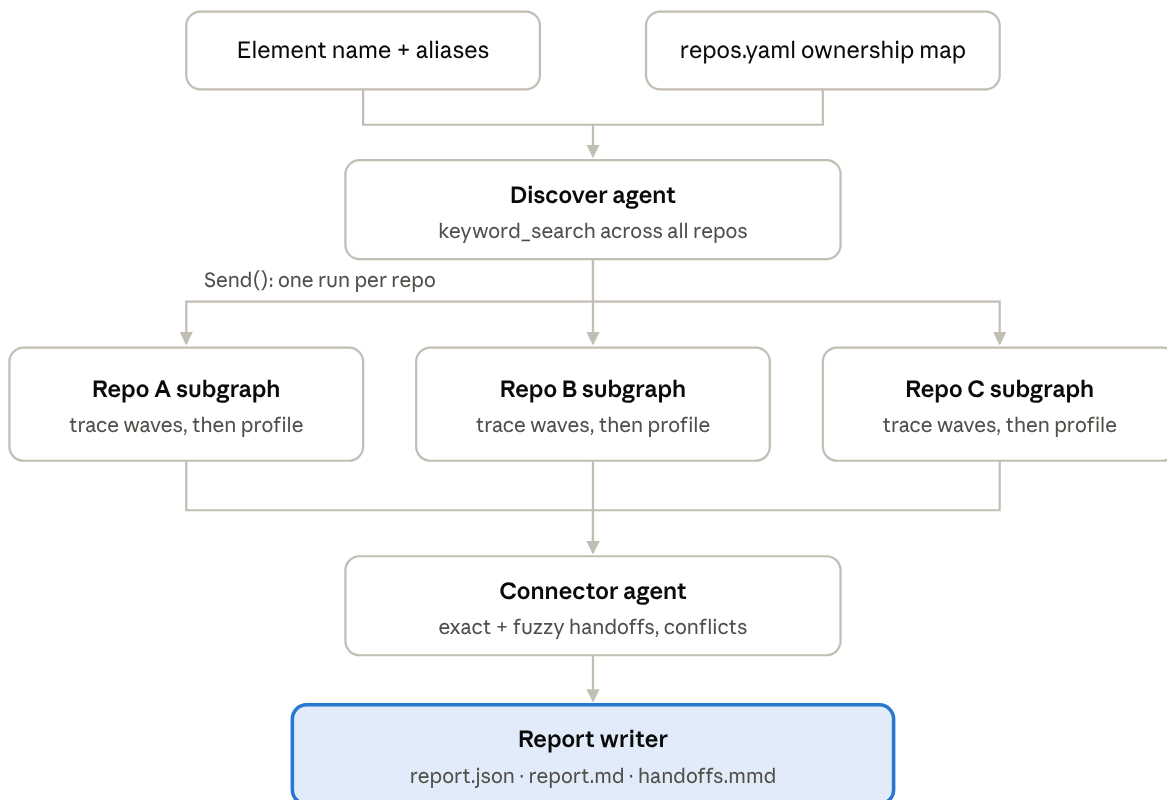
Tutorial: build a repo-segregated lineage tracer with LangChain and Sourcegraph MCP

Sep 26, 2026 · @Rajat Mehta

What you'll build

You'll build a command-line tool. You give it a business element name like `net_revenue`, and it reports, repository by repository, where that element is calculated, used, stored, and ingested. Every calculation comes with a citation pinned to a commit, plus its inputs traced up to where they're ingested and its output traced down to where it's stored.

Lineage tracer: one isolated run per repository



architecture · discover, one subgraph per repo, connector, report

Discover finds the element everywhere. Each repository is then traced in its own isolated run, and the connector links the repos through the tables, topics, and APIs they share.

Component	Kind	What it does	File
Discover agent	LLM + tools	Finds every occurrence with <code>keyword_search</code> and groups the hits by repo	<code>main_graph.py</code>
Repo subgraph	LangGraph loop	Runs the upstream, downstream, and storage tracers in waves, fenced inside one repo	<code>repo_graph.py</code>
Repo profiler	LLM, no tools	Turns one repo's findings into calculations with citations, inputs, and outputs	<code>repo_graph.py</code>
Connector agent	Code + LLM, no tools	Matches one repo's outgoing boundaries to another repo's incoming ones; flags conflicting formulas	<code>main_graph.py</code>
Report writer	Plain code	Writes JSON, Markdown, and a Mermaid system map, with Sourcegraph links	<code>report.py</code>

Stack: LangChain `create_agent` with its middleware harness, LangGraph for orchestration, `langchain-mcp-adapters` for Sourcegraph MCP, and pytest for tests. Search uses **keyword_search only**, with no `nls_search`.

Prerequisites and setup

You need Python 3.10+, a Sourcegraph Enterprise instance with MCP enabled, a Sourcegraph access token, and an Anthropic API key. Nothing in this build needs `nls_search`. All searching goes through `keyword_search`, backed by `read_file`, `go_to_definition`, and `find_references`.

Sourcegraph tool	Needed?	Used for
<code>keyword_search</code>	Required	Finding occurrences, assignments, reads and writes
<code>read_file</code>	Required	Reading code, supplying line numbers, verifying citations

Sourcegraph tool	Needed?	Used for
<code>go_to_definition</code>	Required	Jumping from a usage to where a value is computed
<code>find_references</code>	Required	Finding where a value is used
<code>commit_search</code>	Optional	Pinning citations to a commit; falls back to <code>HEAD</code>
<code>nls_search</code>	Not used	—

Use the `/api/mcp/all` endpoint, which exposes the full tool suite. The core `/api/mcp` endpoint may not include every tool above.

1. Create the project

```
mkdir lineage_tracer_v2 && cd lineage_tracer_v2
mkdir tests golden
python -m venv .venv && source .venv/bin/activate
```

2. Add `requirements.txt` and install

```
langchain>=1.0
langgraph>=1.0
langchain-mcp-adapters
langchain-anthropic
pydantic>=2
pyyaml
pytest
```

```
pip install -r requirements.txt
```

3. Add `pytest.ini`, so tests can import the modules from the project root:

```
[pytest]
pythonpath = .
testpaths = tests
```

4. Set your environment

```

export SRC_ENDPOINT=https://sourcegraph.yourcompany.com
export SRC_ACCESS_TOKEN=sgp_xxx          # a token with the mcp scope is
enough
export ANTHROPIC_API_KEY=sk-ant-xxx
# optional
export LINEAGE_REVISION=main             # pin every citation to this branch
or commit

```

Final layout (every file is built in the steps below):

```

lineage_tracer_v2/
  config.py      repos.yaml      models.py      prompts.py
  agents.py      citations.py    linking.py
  repo_graph.py  main_graph.py  report.py      run.py
  check_tools.py evaluate.py    golden/net_revenue.example.yaml
  tests/__init__.py tests/fakes.py tests/test_citations.py
  tests/test_linking.py tests/test_graph_offline.py

```

Step 1: configuration and the ownership map

This step creates two files. `config.py` holds the endpoints and the safety limits that keep a run cheap. `repos.yaml` says which system and team own each repository, which is how every section of the report gets an owner.

config.py

```

"""Step 1 - Configuration: endpoints, limits, and the repo ownership map."""
import os
from pathlib import Path

import yaml

MODEL = os.getenv("LINEAGE_MODEL", "anthropic:claude-sonnet-5")
SUMMARY_MODEL = os.getenv("LINEAGE_SUMMARY_MODEL", "anthropic:claude-haiku-4-5-20251001")

SG_ENDPOINT = os.getenv("SRC_ENDPOINT", "").rstrip("/")
SG_URL = SG_ENDPOINT + "/.api/mcp/all"
SG_TOKEN = os.getenv("SRC_ACCESS_TOKEN", "")
REVISION = os.getenv("LINEAGE_REVISION", "") # pin a branch or commit;
empty = auto

```

```
MAX_DEPTH = int(os.getenv("LINEAGE_MAX_DEPTH", "3"))           # trace waves per
repo
MAX_SYMBOLS = int(os.getenv("LINEAGE_MAX_SYMBOLS", "8"))        # symbols per
wave per repo
MAX_REPOS = int(os.getenv("LINEAGE_MAX_REPOS", "10"))           # repos traced
per run
CONCURRENCY = int(os.getenv("LINEAGE_CONCURRENCY", "4"))        # parallel LLM
calls
TOOL_CALL_LIMIT = int(os.getenv("LINEAGE_TOOL_CALL_LIMIT", "25")) # per
agent run

def load_repo_catalog(path: str = "repos.yaml") -> dict:
    """Return {repo_name: {"system": ..., "owner": ...}}, or {} if the file
    is missing."""
    p = Path(path)
    if not p.exists():
        return {}
    return yaml.safe_load(p.read_text()) or {}
```

repos.yaml : the keys must match Sourcegraph repo names exactly.

```
github.com/acme/billing-service:
  system: Billing
  owner: "@payments-team"
github.com/acme/data-pipeline:
  system: Analytics warehouse
  owner: "@data-eng"
github.com/acme/finance-dashboard:
  system: Finance reporting
  owner: "@finance-apps"
```

Setting	Default	Raise it when	Lower it when
LINEAGE_MAX_DEPTH	3	Chains of calculations are long	A first run, or cost matters
LINEAGE_MAX_SYMBOLS	8	A repo has many real definitions	Discovery returns noise
LINEAGE_MAX_REPOS	10	The element spans many systems	Testing on one or two repos

Setting	Default	Raise it when	Lower it when
LINEAGE_CONCURRENCY	4	Your rate limits are generous	You see 429 errors
LINEAGE_TOOL_CALL_LIMIT	25	Agents stop before finishing	Agents wander

Step 2: data models with citations, inputs, and outputs

Agents never write free text. They fill in these Pydantic models, which `create_agent` validates through `response_format`. The key design choice is that agents only supply a `Loc`: a path plus line numbers copied from `read_file`. Code turns each `Loc` into a pinned citation with a link in Step 5, so the LLM never invents a URL or a commit.

Model	Filled by	Holds
Symbol	Discover, tracers	A place to trace: repo, path, name
Loc	Every agent	Path, start line, end line
Edge	Tracers	produces / uses / stored_in / ingested_from, with formula and loc
Boundary	Tracers	Where data enters (<code>in</code>) or leaves (<code>out</code>) a repo
CalculationDraft	Repo profiler	Formula + loc, its inputs (origin, source, loc) and its output (stored_in, loc)
Handoff	Connector	Repo A to repo B, via a shared table, topic or API

models.py

```
"""Step 2 - Data shapes. Agents fill these in; code turns them into cited
records."""
from typing import Literal, Optional

from pydantic import BaseModel, Field

class Symbol(BaseModel):
```

```

    repo: str = Field(description="Exact Sourcegraph repo name, e.g.
github.com/acme/billing-service")
    path: str
    name: str
    line: Optional[int] = None

    @property
    def id(self) -> str:
        return f"{self.repo}:{self.path}:{self.name}"

class DiscoveryResult(BaseModel):
    symbols: list[Symbol] = []

class Loc(BaseModel):
    """Where something was seen. Line numbers must be copied from read_file
output."""
    path: str
    start_line: int
    end_line: int

class Edge(BaseModel):
    source: str
    target: str
    relation: Literal["produces", "uses", "stored_in", "ingested_from"]
    access: Literal["write", "read"]
    formula: Optional[str] = Field(None, description="Exact calculation, if
any")
    loc: Loc

class Boundary(BaseModel):
    """A point where data enters or leaves the repository."""
    direction: Literal["in", "out"]
    channel: Literal["table", "topic", "api", "file", "package"]
    name: str = Field(description="Fully qualified:
table:schema.table.column, kafka:topic, GET /api/path")
    carries: list[str] = Field(default_factory=list, description="Fields that
cross this boundary")
    loc: Loc

```

```

class TraceResult(BaseModel):
    edges: list[Edge] = []
    boundaries: list[Boundary] = []
    new_symbols: list[Symbol] = Field(default_factory=list,
                                      description="Symbols in THIS repo that
still need tracing")

class InputDraft(BaseModel):
    name: str
    origin: Literal["ingested", "computed", "constant", "unresolved"]
    source: Optional[str] = Field(None, description="ingested: the boundary
name; computed: the variable it is computed as")
    loc: Optional[Loc] = Field(None, description="Where this input is read or
defined")

class OutputDraft(BaseModel):
    name: str
    stored_in: list[str] = Field(default_factory=list, description="Boundary
names the result is written to")
    loc: Optional[Loc] = Field(None, description="Where the result is written
or returned")

class CalculationDraft(BaseModel):
    formula: str
    loc: Loc = Field(description="The lines where the calculation happens")
    inputs: list[InputDraft] = []
    output: OutputDraft

class RepoProfile(BaseModel):
    role: Literal["producer", "consumer", "producer+consumer", "source of
inputs", "pass-through"]
    summary: str
    calculations: list[CalculationDraft] = []

class Handoff(BaseModel):
    from_repo: str
    to_repo: str

```

```
via: str
carries: list[str] = []
match: Literal["exact", "fuzzy"] = "fuzzy"

class ConnectorResult(BaseModel):
    extra_handoffs: list[Handoff] = []
    conflicts: list[str] = []
```

The final calculation record that code builds from a `CalculationDraft` looks like this (Step 5 builds it):

```
{
  "id": "calc:data-pipeline:models/fct_orders.sql:42",
  "formula": "gross_amount - discount_amount - refund_amount",
  "citation": {"repo": "...", "path": "...", "start_line": 42, "end_line":
42,
              "commit": "<sha>", "url": "https://.../-/blob/...?L42-42",
"verified": true},
  "inputs": [{"name": "gross_amount", "origin": "ingested", "source":
"kafka:orders.v2",
              "citation": {...}, "computed_by": []}],
  "output": {"name": "net_revenue", "stored_in":
["table:analytics.fct_orders.net_revenue"],
              "citation": {...}},
  "feeds": ["calc:finance-dashboard:metrics/revenue.py:88"]
}
```

`computed_by` traces an input up to the calculation that produced it. `feeds` traces the output down to the calculations that use it. Both are filled in by code in Step 7.

Step 3: agent prompts using keyword_search only

Without `nls_search`, agents can't ask "where is revenue calculated?" in plain language. They have to search for literal text. So every prompt carries the same `SEARCH_RECIPES`: the alias spellings, noise filters, and write and read patterns that stand in for semantic search. `scope_rules()` fences each tracer inside one repository at one revision.

Prompt	Agent	Job
DISCOVERY_PROMPT	Discover	Find the element in every repo

Prompt	Agent	Job
<code>scope_rules(repo, revision)</code>	All tracers	<code>repo: ^name\$</code> on every search, read at the pinned revision, stop at boundaries
<code>UPSTREAM_PROMPT</code>	Upstream tracer	How the element is produced, with formula and loc
<code>DOWNSTREAM_PROMPT</code>	Downstream tracer	Where it is used
<code>STORAGE_PROMPT</code>	Storage tracer	Where it and its inputs are stored or ingested
<code>REPO_PROFILER_PROMPT</code>	Repo profiler	Turn findings into calculations with inputs and output
<code>CONNECTOR_PROMPT</code>	Connector	Fuzzy handoffs and conflicting formulas

`prompts.py`

```

"""Step 3 - What each agent is told. Search is keyword_search only (no
nls_search)."""
import re

from config import MAX_SYMBOLS

GROUNDING = ("Every path and line number you return MUST be copied from a
read_file result. "
             "Never guess. If you cannot confirm something, leave it out.")

SEARCH_RECIPES = """You only have keyword_search (no semantic search), so
search literally:
- Each alias on its own: net_revenue, netRevenue, NET_REVENUE, net-revenue.
- Add filters to cut noise: lang:sql, lang:python, lang:java, -file:test, -
file:mock, -file:fixture.
- Assignments: "net_revenue =", "AS net_revenue", "netRevenue:", "netRevenue
=".
- Writes: INSERT INTO, to_sql(, .write, produce(, publish(, .save(, @Column.
- Reads: FROM, read_sql, consume(, subscribe(, requests.get, @GetMapping.
- Pair a write/read recipe with the element or table name, or restrict to a
file with file: ^path$.
Open promising hits with read_file before you report them."""

```

```
DISCOVERY_PROMPT = f"""You locate every place a business element appears in
code, across ALL repositories.
{SEARCH_RECIPES}
Keep assignments, calculations, SQL columns/aliases, model fields and
meaningful reads.
Skip tests, mocks, fixtures and vendored code.
Use the exact repository name Sourcegraph returns (e.g.
github.com/acme/billing-service).
Return at most {MAX_SYMBOLS} symbols per repository. {GROUNDING}"""
```

```
def scope_rules(repo: str, revision: str) -> str:
    """Rules that fence an agent inside ONE repository at ONE revision."""
    rev_rule = (" if revision == \"HEAD\" else
                f\"\\n- Pass revision `{revision}` to read_file,
go_to_definition and find_references.\")
    return f"""SCOPE: you work ONLY inside repository `{repo}`.
- Add `repo:^(re.escape(repo))$` to every keyword_search query.{rev_rule}
- Only read files and follow symbols in this repository.
- When data ENTERS or LEAVES this repository (table/column, queue topic, API
endpoint,
  file, or a package from another repo), record a Boundary with a fully
qualified name
  and DO NOT follow it further.
- Every Edge and Boundary needs a loc: path, start_line, end_line from
read_file.
{SEARCH_RECIPES}
- {GROUNDING}"""
```

```
UPSTREAM_PROMPT = """You find how the element is PRODUCED in this repository.
1. Open each place where it is assigned or computed (read_file;
go_to_definition if you only have a usage).
2. Record edge input -> element, relation 'produces', access 'write', with
the exact formula and its loc.
3. If a value simply arrives from outside the repo, record an 'in' Boundary
instead.
4. Put inputs that are themselves computed in this repo into new_symbols."""
```

```
DOWNSTREAM_PROMPT = """You find where the element is USED in this repository.
1. Call find_references on its definition. Ignore references in other
repositories;
```

```

    if there are some, record one 'out' Boundary with channel 'package'.
2. read_file around each in-repo reference.
3. Record edge element -> consumer, relation 'uses', access 'read', with its
loc. If the consumer
    transforms the value, include the formula and add the consumer to
new_symbols."""

```

```

STORAGE_PROMPT = """You find how the element and its inputs are STORED or
INGESTED in this repository.
Look at SQL, dbt models, ORM models, migrations, ETL/Airflow jobs, queue
producers and
consumers, API handlers, and file readers/writers.
- Writing to a table/topic/file/API -> 'out' Boundary plus a 'stored_in' edge
(access 'write').
- Reading from one -> 'in' Boundary plus an 'ingested_from' edge (access
'read').
Always use fully qualified names: table:schema.table.column, kafka:topic, GET
/api/path."""

```

```

REPO_PROFILER_PROMPT = """You turn ONE repository's findings into calculation
records,
using ONLY the edges and boundaries given to you.
- role: producer (computes the element), consumer (only reads it),
producer+consumer,
    source of inputs (supplies its inputs but never computes it), or pass-
through (moves it unchanged).
- calculations: one per DISTINCT formula found in 'produces' edges. For each:
    - formula and loc: copied from the edge.
    - inputs: every variable in the formula. origin = 'ingested' if an 'in'
boundary or an
    'ingested_from' edge carries it (source = that boundary name); 'computed'
if another
    'produces' edge creates it (source = its name); 'constant' for literals;
else 'unresolved'.
    loc = where the input is read or defined, if an edge or boundary gives
it.
    - output: the variable/column the formula assigns. stored_in = names of
'out' boundaries or
    'stored_in' edges that write it; loc = where it is written.
- summary: one or two plain sentences.
Copy every loc exactly. Never add facts that are not in the input."""

```

```

CONNECTOR_PROMPT = """You connect per-repository lineage reports for one

```

```
business element.
Input: each repo's role, calculations and boundaries, plus handoffs already
matched by exact name.
1. extra_handoffs: add links where one repo's 'out' boundary and another
repo's 'in' boundary
    clearly refer to the same table/topic/API but are named slightly
differently
    (e.g. fct_orders vs analytics.fct_orders). Do not repeat exact matches.
Set match='fuzzy'.
2. conflicts: where repos compute the element with DIFFERENT formulas, write
one sentence
    naming the repos and the difference.
Only use what is in the input."""
```

Tune the recipes to your code. The recipes are the main lever you have without semantic search. Add the write and read idioms your teams actually use, like your ORM's save method or your internal Kafka wrapper. Discovery and storage tracing improve immediately.

Step 4: agents, the MCP connection, and the harness

The tool-using agents (discover and the three tracers) are built with `create_agent`. That gives you the harness: the loop where the model calls tools and reads results, tool execution, and structured output. Middleware adds retries, a cost cap, and context compression. The profiler and connector need no tools, so each is a single `with_structured_output` call.

Middleware	Why it's here
<code>ModelRetryMiddleware</code>	Many agents run in parallel, so rate-limit and model errors happen
<code>ToolRetryMiddleware</code>	Sourcegraph calls occasionally time out
<code>ToolCallLimitMiddleware</code>	Caps each agent at <code>TOOL_CALL_LIMIT</code> tool calls, so none spins forever
<code>SummarizationMiddleware</code>	Compresses history when <code>read_file</code> output fills the context

`agents.py`

```

"""Step 4 - Connect Sourcegraph MCP and build the agents with the
create_agent harness."""
import asyncio

from langchain.agents import create_agent
from langchain.agents.middleware import (ModelRetryMiddleware,
SummarizationMiddleware,
                                         ToolCallLimitMiddleware,
                                         ToolRetryMiddleware)
from langchain.chat_models import init_chat_model
from langchain_mcp_adapters.client import MultiServerMCPClient

from config import CONCURRENCY, MODEL, SG_TOKEN, SG_URL, SUMMARY_MODEL,
TOOL_CALL_LIMIT
from models import ConnectorResult, DiscoveryResult, RepoProfile, TraceResult
from prompts import (DISCOVERY_PROMPT, DOWNSTREAM_PROMPT, STORAGE_PROMPT,
                     UPSTREAM_PROMPT, scope_rules)

_limit = asyncio.Semaphore(CONCURRENCY)

async def load_sourcegraph_tools():
    client = MultiServerMCPClient({
        "sourcegraph": {
            "transport": "streamable_http",
            "url": SG_URL,
            "headers": {"Authorization": f"token {SG_TOKEN}"},
        }
    })
    return await client.get_tools()

def pick(tools, *names):
    """Give an agent only the Sourcegraph tools it needs (matched by
substring)."""
    return [t for t in tools if any(n in t.name for n in names)]

def tool_named(tools, name):
    return next((t for t in tools if name in t.name), None)

def harness():

```

```

    """Middleware for every tool-using agent: retries, a cost cap, context
compression."""
    return [
        ModelRetryMiddleware(max_retries=3),
        ToolRetryMiddleware(max_retries=2),
        ToolCallLimitMiddleware(run_limit=TOOL_CALL_LIMIT,
exit_behavior="end"),
        SummarizationMiddleware(model=SUMMARY_MODEL, trigger=("tokens",
80000),
                                keep=("messages", 12)),
    ]

class Agents:
    def __init__(self, tools):
        self.tools = tools
        llm = init_chat_model(MODEL)

        self.discovery = create_agent(
            MODEL, tools=pick(tools, "keyword_search", "read_file"),
            system_prompt=DISCOVERY_PROMPT, response_format=DiscoveryResult,
            middleware=harness())

        # No tools: they only reason over what the tracers found.
        self.repo_profiler = llm.with_structured_output(RepoProfile)
        self.connector = llm.with_structured_output(ConnectorResult)

    def tracers_for(self, repo: str, revision: str) -> dict:
        """Three tracer agents fenced inside one repository at one
revision."""
        rules = scope_rules(repo, revision)
        specs = {
            "upstream": (UPSTREAM_PROMPT, ("go_to_definition", "read_file",
"keyword_search")),
            "downstream": (DOWNSTREAM_PROMPT, ("find_references",
"read_file", "keyword_search")),
            "storage": (STORAGE_PROMPT, ("keyword_search", "read_file")),
        }
        return {
            role: create_agent(MODEL, tools=pick(self.tools, *tool_names),
                                system_prompt=f"{prompt}\n\n{rules}",
                                response_format=TraceResult,
middleware=harness())

```

```

        for role, (prompt, tool_names) in specs.items()
    }

async def run_agent(agent, text: str):
    """Run a tool-using agent and return its structured answer."""
    async with _limit:
        out = await agent.ainvoke({"messages": [{"role": "user", "content":
text}]}],
                                config={"recursion_limit": 60})
    return out["structured_response"]

async def run_llm(structured_llm, system: str, text: str):
    """Run a no-tools reasoning step and return its structured answer."""
    async with _limit:
        return await structured_llm.ainvoke([{"role": "system", "content":
system},
                                            {"role": "user", "content":
text}])

```

`pick()` matches tool names by substring, so it works whether your server calls the tool `keyword_search` or `sg_keyword_search`. Every agent call goes through `run_agent` or `run_llm`. That makes them the two seams the offline tests replace with fakes in the testing section.

Step 5: citations you can trust

Every calculation, input, and output gets a citation that is pinned, linkable, and verified. This file does three jobs, all in plain code rather than the LLM:

1. **Pin:** `resolve_revision` uses `LINEAGE_REVISION` if you set it. Otherwise it takes the newest commit sha from `commit_search`, falling back to `HEAD`.
2. **Link:** `make_citation` turns an agent's `Loc` into `{repo, path, lines, commit, url}`. It rejects impossible line numbers, and `finalize_calculation` drops any calculation whose main citation is invalid.
3. **Verify:** `verify_calculation` re-reads each cited file with `read_file` and checks that the output name or a formula identifier really appears within 2 lines of the cited range. Each citation ends up `verified: true`, `false`, or `null` (could not check).

`citations.py`

```

"""Step 5 - Citations: pin a revision, build Sourcegraph links, verify lines
against the file."""
import re
from urllib.parse import quote

from config import REVISION, SG_ENDPOINT

SHA_RE = re.compile(r"\b[0-9a-f]{40}\b")
LINE_RE = re.compile(r"^\s*(\d+)[\s:|]\s?(.*)$")
IDENT_RE = re.compile(r"[A-Za-z_][A-Za-z0-9_]{2,}")

def as_text(out) -> str:
    """MCP tool results may be a string or a list of content blocks."""
    if isinstance(out, str):
        return out
    if isinstance(out, list):
        return "\n".join(b.get("text", "") if isinstance(b, dict) else str(b)
        for b in out)
    return str(out)

async def resolve_revision(tools, repo: str) -> str:
    """LINEAGE_REVISION if set; else the newest commit sha commit_search
    returns; else HEAD."""
    if REVISION:
        return REVISION
    tool = next((t for t in tools if "commit_search" in t.name), None)
    if tool:
        try:
            m = SHA_RE.search(as_text(await tool.ainvoke({"repos": [repo]})))
            if m:
                return m.group(0)
        except Exception as e:
            print(f" [{repo}] could not resolve commit ({e}); using HEAD")
    return "HEAD"

def build_url(repo: str, revision: str, path: str, start: int, end: int) ->
str:
    rev = "" if revision in ("", "HEAD") else f"@{revision}"
    return f"{SG_ENDPOINT}/{repo}{rev}/-/blob/{quote(path)}?L{start}-{end}"

```

```

def make_citation(repo: str, revision: str, loc: dict | None) -> dict | None:
    """Turn an agent's loc into a pinned, linkable citation. Bad line numbers
    -> None."""
    if not loc or not loc.get("path"):
        return None
    try:
        start = int(loc["start_line"])
        end = int(loc.get("end_line") or start)
    except (KeyError, TypeError, ValueError):
        return None
    if start < 1 or end < start:
        return None
    return {"repo": repo, "path": loc["path"], "start_line": start,
"end_line": end,
        "commit": revision, "url": build_url(repo, revision, loc["path"],
start, end),
        "verified": None}

def finalize_calculation(repo: str, revision: str, draft: dict) -> dict |
None:
    """Agent draft -> cited calculation record. Drops drafts without a valid
    main citation."""
    cite = make_citation(repo, revision, draft.get("loc"))
    if cite is None:
        return None
    short = repo.rstrip("/").split("/")[-1]
    out = draft["output"]
    return {
        "id": f"calc:{short}:{cite['path']}:{cite['start_line']}",
        "formula": draft["formula"],
        "citation": cite,
        "inputs": [{"name": i["name"], "origin": i["origin"], "source":
i.get("source"),
                    "citation": make_citation(repo, revision, i.get("loc")),
"computed_by": []}
                    for i in draft.get("inputs", [])],
        "output": {"name": out["name"], "stored_in": out.get("stored_in",
[]),
                    "citation": make_citation(repo, revision,
out.get("loc"))},
        "feeds": [],

```

```

}

def numbered_lines(text: str) -> dict[int, str]:
    """Parse read_file output into {line_number: text}, numbered or not."""
    raw = text.splitlines()
    matches = [LINE_RE.match(line) for line in raw]
    non_empty = [line for line in raw if line.strip()]
    if non_empty and sum(1 for m in matches if m) >= 0.6 * len(non_empty):
        return {int(m.group(1)): m.group(2) for m in matches if m}
    return {i: line for i, line in enumerate(raw, 1)}

def supports(lines: dict[int, str], start: int, end: int, names: list[str],
slack: int = 2) -> bool:
    """True if any of the names appears within the cited lines (+/-
slack)."""
    window = " ".join(lines.get(i, "") for i in range(max(1, start - slack),
end + slack + 1))
    return any(re.search(rf"\b{re.escape(n)}\b", window, re.IGNORECASE) for n
in names if n)

async def verify_calculation(read_tool, calc: dict, revision: str, cache:
dict) -> None:
    """Re-read each cited file and mark citations verified True/False (None =
could not check)."""

    async def lines_for(cite):
        key = (cite["repo"], cite["path"])
        if key not in cache:
            args = {"repo": cite["repo"], "path": cite["path"]}
            if revision not in ("", "HEAD"):
                args["revision"] = revision
            try:
                cache[key] = numbered_lines(as_text(await
read_tool.ainvoke(args)))
            except Exception as e:
                print(f" could not read {key}: {e}")
                cache[key] = None
        return cache[key]

    async def check(cite, names):

```

```

    if cite:
        lines = await lines_for(cite)
        cite["verified"] = None if lines is None else supports(
            lines, cite["start_line"], cite["end_line"], names)

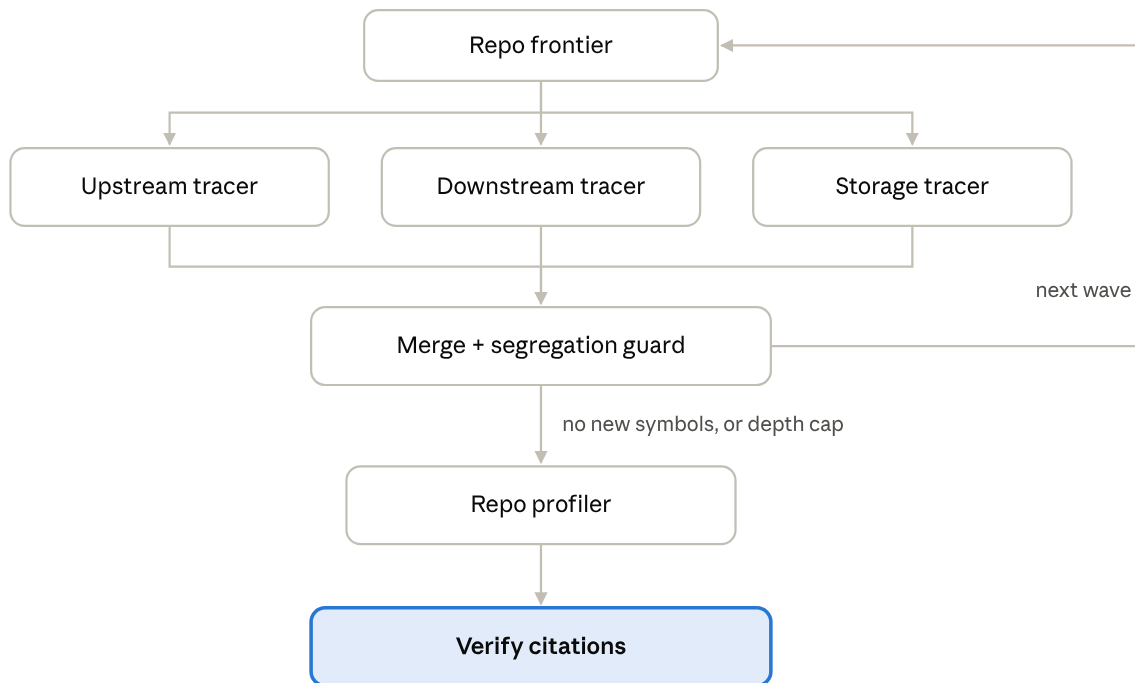
    await check(calc["citation"], [calc["output"]["name"],
*IDENT_RE.findall(calc["formula"])]))
    for inp in calc["inputs"]:
        await check(inp["citation"], [inp["name"]])
    await check(calc["output"]["citation"], [calc["output"]["name"]])

```

About the commit pin: `commit_search` isn't guaranteed to return the newest commit first on every instance. For audit-grade citations, set `LINEAGE_REVISION` to a release tag or commit sha. Each citation records the revision it was read at, so a report stays reproducible after the code moves on.

Step 6: the per-repository subgraph

Each repository gets its own copy of this graph, fenced to that repo and pinned to one revision. It runs trace waves until nothing new turns up, then profiles the findings into calculations, then verifies every citation.

Inside one repository subgraph

repo subgraph · trace waves, profile, verify

The three tracers run in parallel on every symbol in the frontier. The merge step deduplicates the results and drops any symbol from another repo. That drop is the segregation guard, and it holds even if an agent ignores its prompt.

repo_graph.py

```

"""Step 6 - The per-repository subgraph: trace waves -> profile -> verify
citations."""
import asyncio
import json

from langgraph.graph import END, START, StateGraph
from typing_extensions import TypedDict

from agents import Agents, run_agent, run_llm, tool_named
from citations import finalize_calculation, verify_calculation
from config import MAX_DEPTH, MAX_SYMBOLS
from models import RepoProfile
from prompts import REPO_PROFILER_PROMPT

```

```

class RepoState(TypedDict):
    element: str
    repo: str
    revision: str
    frontier: list[dict]      # symbols to trace in the next wave
    visited: list[str]
    edges: list[dict]
    boundaries: list[dict]
    depth: int
    role: str
    summary: str
    calculations: list[dict]

def sid(s: dict) -> str:
    return f"{s['repo']}:{s['path']}:{s['name']}"

def build_repo_graph(agents: Agents, repo: str, revision: str, verify: bool =
True):
    tracers = agents.tracers_for(repo, revision)
    read_tool = tool_named(agents.tools, "read_file")

    async def trace(state: RepoState):
        jobs = [(role, s) for s in state["frontier"] for role in tracers]
        results = await asyncio.gather(*[
            run_agent(tracers[role],
                    f"Business element: {state['element']}\nSymbol to
trace: {json.dumps(s)}")
            for role, s in jobs
        ], return_exceptions=True)

        edges, boundaries = list(state["edges"]), list(state["boundaries"])
        seen_e = {(e["source"], e["target"], e["relation"]) for e in edges}
        seen_b = {(b["direction"], b["name"].lower()) for b in boundaries}
        visited = set(state["visited"]) | {sid(s) for s in state["frontier"]}
        next_wave = {}

        for (role, _), r in zip(jobs, results):
            if isinstance(r, Exception):
                print(f" [{repo}] {role} failed: {r}")
                continue
            for e in r.edges:

```

```

        key = (e.source, e.target, e.relation)
        if key not in seen_e:
            seen_e.add(key)
            edges.append(**e.model_dump(), "found_by": role})
    for b in r.boundaries:
        key = (b.direction, b.name.lower())
        if key not in seen_b:
            seen_b.add(key)
            boundaries.append(b.model_dump())
    for s in r.new_symbols:
        if s.repo != repo:                # segregation guard: never leave
this repo
            continue
        if s.id not in visited:
            next_wave[s.id] = s.model_dump()

    frontier = list(next_wave.values())[0:MAX_SYMBOLS]
    print(f" [{repo}] wave {state['depth'] + 1}: {len(edges)} edges, "
          f"{len(boundaries)} boundaries, {len(frontier)} queued")
    return {"edges": edges, "boundaries": boundaries, "visited":
sorted(visited),
        "frontier": frontier, "depth": state["depth"] + 1}

def keep_going(state: RepoState):
    return "trace" if state["frontier"] and state["depth"] < MAX_DEPTH
else "profile"

async def profile(state: RepoState):
    payload = json.dumps({"repo": repo, "element": state["element"],
                          "edges": state["edges"], "boundaries":
state["boundaries"]},
                        indent=1)
    prof: RepoProfile = await run_llm(agents.repo_profiler,
REPO_PROFILER_PROMPT, payload)
    calcs = [finalize_calculation(repo, state["revision"],
c.model_dump())
              for c in prof.calculations]
    return {"role": prof.role, "summary": prof.summary,
            "calculations": [c for c in calcs if c]}

async def verify_citations(state: RepoState):
    if not (verify and read_tool):
        return {}

```

```

        cache: dict = {}
        calcs = [dict(c) for c in state["calculations"]]
        for c in calcs:
            await verify_calculation(read_tool, c, state["revision"], cache)
        ok = sum(1 for c in calcs if c["citation"]["verified"])
        print(f" [{repo}] citations verified: {ok}/{len(calcs)}
calculations")
        return {"calculations": calcs}

g = StateGraph(RepoState)
g.add_node("trace", trace)
g.add_node("profile", profile)
g.add_node("verify", verify_citations)
g.add_edge(START, "trace")
g.add_conditional_edges("trace", keep_going, ["trace", "profile"])
g.add_edge("profile", "verify")
g.add_edge("verify", END)
return g.compile()

```

How "trace up and down" happens here: the upstream tracer follows each input back to where it is computed or ingested (an `in` boundary). The storage tracer follows the output forward to where it is written (an `out` boundary). New computed inputs join the next wave, so chains of calculations get walked hop by hop until they reach the repo's edge.

Step 7: linking and the main graph

The main graph runs `discover` once, launches one repo subgraph per repository in parallel with `Send`, waits for all of them, and then connects them. Linking happens in plain code, so it's deterministic and testable:

- `match_boundaries` pairs one repo's `out` boundary with another repo's `in` boundary by exact name. Each pair is a handoff.
- `link_calculations` traces up and down. It fills `inputs[].computed_by` with the calculation whose output has the same normalized name (`net_revenue` = `netRevenue`), preferring the same repo. It only crosses into another repo when a handoff connects the two. The reverse links fill `feeds`.
- `trace_chain` walks the whole chain, upstream or downstream, from any calculation.

The connector LLM only adds what exact matching can't: near-miss names (`fct_orders` vs `analytics.fct_orders`) and one-line explanations of conflicting formulas.

linking.py

```
"""Step 7 - Link repos (boundaries) and calculations (inputs <-> outputs) in
plain code."""
```

```
import copy
import re
```

```
def norm(name: str) -> str:
    """net_revenue, netRevenue and NET-REVENUE all become netrevenue."""
    return re.sub(r"^[a-z0-9]", "", (name or "").lower())
```

```
def match_boundaries(reports: list[dict]) -> list[dict]:
    """Exact-name match of one repo's 'out' boundary to another repo's 'in'
    boundary."""
    outs = [(r["repo"], b) for r in reports for b in r["boundaries"] if
b["direction"] == "out"]
    ins = [(r["repo"], b) for r in reports for b in r["boundaries"] if
b["direction"] == "in"]
    return [
        {"from_repo": src, "to_repo": dst, "via": f"{ob['channel']}:
{ob['name']}",
        "carries": ob.get("carries", []), "match": "exact"}
        for src, ob in outs for dst, ib in ins
        if src != dst and ob["name"].lower() == ib["name"].lower()
    ]
```

```
def link_calculations(reports: list[dict], handoffs: list[dict]) ->
list[dict]:
    """Fill inputs[].computed_by (trace up) and feeds (trace down). Returns
    new reports.
```

```
    An input links to a calculation whose output has the same (normalised)
    name,
    in the same repo first; across repos only when a handoff connects the two
    repos.
```

```
    """
    reports = copy.deepcopy(reports)
    connected = {(h["from_repo"], h["to_repo"]) for h in handoffs}
    producers: dict[str, list[tuple[str, dict]]] = {}
    for r in reports:
```

```

        for c in r["calculations"]:
            producers.setdefault(norm(c["output"]["name"]),
                                []).append((r["repo"], c))

    for r in reports:
        for c in r["calculations"]:
            for inp in c["inputs"]:
                candids = [(repo, p) for repo, p in
producers.get(norm(inp["name"]), [])
                if p["id"] != c["id"]]
                same = [p for repo, p in candids if repo == r["repo"]]
                cross = [p for repo, p in candids if (repo, r["repo"]) in
connected]

                inp["computed_by"] = [p["id"] for p in (same or cross)]
                for p in (same or cross):
                    if c["id"] not in p["feeds"]:
                        p["feeds"].append(c["id"])

    return reports

def trace_chain(reports: list[dict], calc_id: str, direction: str = "up") ->
list[str]:
    """Every calculation upstream ('up') or downstream ('down') of calc_id,
nearest first."""
    index = {c["id"]: c for r in reports for c in r["calculations"]}

    def nxt(cid):
        c = index[cid]
        if direction == "up":
            return [p for inp in c["inputs"] for p in inp["computed_by"]]
        return list(c["feeds"])

    seen, queue, order = {calc_id}, [calc_id], []
    while queue:
        for n in nxt(queue.pop(0)):
            if n not in seen and n in index:
                seen.add(n)
                order.append(n)
                queue.append(n)
    return order

```

main_graph.py

```

"""Step 7 - The main graph: discover -> one subgraph per repo -> connector ->
report."""
import json
import operator
from typing import Annotated

from langgraph.graph import END, START, StateGraph
from langgraph.types import Send
from typing_extensions import TypedDict

from agents import Agents, run_agent, run_llm
from citations import resolve_revision
from config import MAX_REPOS, MAX_SYMBOLS
from linking import link_calculations, match_boundaries
from prompts import CONNECTOR_PROMPT
from repo_graph import build_repo_graph
from report import write_reports


class MainState(TypedDict):
    element: str
    aliases: list[str]
    verify: bool
    by_repo: dict[str, list[dict]]
    repo_reports: Annotated[list[dict], operator.add]  # each repo appends
its own report
    handoffs: list[dict]
    conflicts: list[str]


class RepoJob(TypedDict):
    element: str
    repo: str
    symbols: list[dict]
    verify: bool


def build_main_graph(agents: Agents, catalog: dict, out_dir: str):

    async def discover(state: MainState):
        res = await run_agent(agents.discovery,
                               f"Business element: {state['element']}\n"
                               f"Aliases: {' '.join(state['aliases'])}")

```

```

        by_repo: dict[str, list[dict]] = {}
        for s in res.symbols:
            by_repo.setdefault(s.repo, []).append(s.model_dump())
        by_repo = {r: syms[:MAX_SYMBOLS] for r, syms in list(by_repo.items())
[:MAX_REPOS]}
        print(f"[discover] found the element in {len(by_repo)} repos:
{list(by_repo)}")
        return {"by_repo": by_repo}

    def fan_out(state: MainState):
        if not state["by_repo"]:
            return ["report"]
        return [Send("repo_trace", {"element": state["element"], "repo": r,
            "symbols": syms, "verify":
state["verify"]})
            for r, syms in state["by_repo"].items()]

    async def repo_trace(job: RepoJob):
        revision = await resolve_revision(agents.tools, job["repo"])
        print(f"[repo] tracing {job['repo']} @ {revision[:12]}")
        graph = build_repo_graph(agents, job["repo"], revision,
job["verify"])
        final = await graph.ainvoke({
            "element": job["element"], "repo": job["repo"], "revision":
revision,
            "frontier": job["symbols"], "visited": [], "edges": [],
"boundaries": [],
            "depth": 0, "role": "", "summary": "", "calculations": [],
        })
        meta = catalog.get(job["repo"], {})
        return {"repo_reports": [{
            "repo": job["repo"],
            "system": meta.get("system", "unknown - add to repos.yaml"),
            "owner": meta.get("owner", "unknown"),
            "revision": revision,
            "role": final["role"],
            "summary": final["summary"],
            "calculations": final["calculations"],
            "boundaries": final["boundaries"], # grounded, straight from
the tracers
            "edges": final["edges"],
        }]}

```

```

    async def connector(state: MainState):
        exact = match_boundaries(state["repo_reports"])
        payload = json.dumps({
            "element": state["element"],
            "repos": [{ "repo": r["repo"], "role": r["role"],
                        "calculations": [{ "formula": c["formula"], "output":
c["output"]["name"]}
                                for c in r["calculations"]],
                        "boundaries": [{k: b[k] for k in ("direction",
"channel", "name")}]
                                for b in r["boundaries"]]}
                        for r in state["repo_reports"]],
            "exact_handoffs": exact,
        }, indent=1)
        res = await run_llm(agents.connector, CONNECTOR_PROMPT, payload)
        return {"handoffs": exact + [h.model_dump() for h in
res.extra_handoffs],
                "conflicts": res.conflicts}

    async def report(state: MainState):
        handoffs = state.get("handoffs", [])
        reports = link_calculations(state.get("repo_reports", []), handoffs)
        write_reports(state["element"], reports, handoffs,
state.get("conflicts", []), out_dir)
        return {}

    g = StateGraph(MainState)
    g.add_node("discover", discover)
    g.add_node("repo_trace", repo_trace)
    g.add_node("connector", connector)
    g.add_node("report", report)
    g.add_edge(START, "discover")
    g.add_conditional_edges("discover", fan_out, ["repo_trace", "report"])
    g.add_edge("repo_trace", "connector")      # waits for every repo to
finish
    g.add_edge("connector", "report")
    g.add_edge("report", END)
    return g.compile()

```

`repo_reports` uses an `operator.add` reducer, so each parallel repo run appends its own report instead of overwriting the others.

Step 8: reports, and running it

The report writer produces four files. Each gives a different view of the same data:

File	For	Shows
<code>report.json</code>	Other tools, <code>evaluate.py</code>	Everything: repos, calculations with citations, inputs, outputs, handoffs, conflicts
<code>report.md</code>	People	One section per repo: owner, role, revision, each calculation with clickable, verified citations
<code>systems.mmd</code>	Owners	Mermaid map of which system hands the element to which
<code>calculations.mmd</code>	Analysts	Mermaid lineage: sources → calculations → stores, grouped per repo

`report.py`

```

"""Step 8 - Write report.json, report.md, systems.mmd and
calculations.mmd."""
import json
import re
from datetime import datetime, timezone
from pathlib import Path

def _cite(c: dict | None) -> str:
    if not c:
        return "no citation"
    mark = {True: "verified", False: "NOT verified", None: "unchecked"}
    [c.get("verified")]
    return f"[{c['path']}:{c['start_line']}-{c['end_line']}]({c['url']})"
    ({mark})"

def _store_key(name: str) -> str:
    """table:analytics.t.c and analytics.t.c are the same store."""
    return "store:" + re.sub(r"^(table|topic|api|file|package):", "",
name.lower())

```

```

def _label(text: str) -> str:
    return str(text).replace("'", '"').replace("|", "/")

def write_reports(element: str, reports: list[dict], handoffs: list[dict],
                  conflicts: list[str], out_dir: str) -> None:
    out = Path(out_dir)
    out.mkdir(parents=True, exist_ok=True)
    reports = sorted(reports, key=lambda r: r["repo"])

    # 1. Machine-readable result
    (out / "report.json").write_text(json.dumps({
        "element": element, "generated_at":
datetime.now(timezone.utc).isoformat(),
        "repositories": reports, "handoffs": handoffs, "conflicts":
conflicts,
    }, indent=2))

    # 2. Human-readable report, one section per repository
    md = [f"# Lineage of `{element}`", ""]
    for r in reports:
        md += [f"## {r['repo']}",
               f"System: {r['system']} | Owner: {r['owner']} | Role:
{r['role'] or '?'} | "
               f"Revision: `{r['revision'][:12]}`", "", r["summary"], ""]
        for c in r["calculations"]:
            md += [f"### `{c['output']['name']}` = {c['formula']}`",
                   f"Calculated at {_cite(c['citation'])}", "", "Inputs:"]
            for i in c["inputs"]:
                src = f" from `{i['source']}`" if i.get("source") else ""
                up = f" <- {'', '.join(i['computed_by'])}" if i["computed_by"]
            else ""
            md.append(f"- `{i['name']}`: {i['origin']}{src}{up} -
{_cite(i['citation'])}")
            o = c["output"]
            stores = "", ".join(f"`{s}`" for s in o["stored_in"])
            where = f"stored in {stores}" if stores else "not stored in this
repo"
            md += [f"Output: `{o['name']}` {where} - {_cite(o['citation'])}"]
            if c["feeds"]:
                md.append(f"Feeds: {'', '.join(c['feeds'])}")

```

```

        md.append("")
        for b in r["boundaries"]:
            verb = "Reads from" if b["direction"] == "in" else "Writes to"
            md.append(f"- {verb} {b['channel']} `{b['name']}` ({b['loc']
['path']}: {b['loc']['start_line']})")
            md.append("")
            md += ["## Handoffs between systems", ""]
            md += [f"- {h['from_repo']} -> {h['to_repo']} via `{h['via']}`
({h['match']})"]
            for h in handoffs] or ["- none found"]
            md += ["", "## Conflicts", ""] + ([f"- {c}" for c in conflicts] or ["-
none found"])
            (out / "report.md").write_text("\n".join(md))

# 3. System map: one node per repo, one arrow per handoff
rid = {r["repo"]: f"r{i}" for i, r in enumerate(reports)}
sysmap = ["flowchart LR"]
for r in reports:
    sysmap.append(f'    {rid[r["repo"]]}["{_label(r["system"])}<br/>'
                    f'({_label(r["repo"].split("/")[-1])}<br/>'
{_label(r["role"])}"]')
    for h in handoffs:
        if h["from_repo"] in rid and h["to_repo"] in rid:
            sysmap.append(f'    {rid[h["from_repo"]]} -->|{_label(h["via"])}|
{rid[h["to_repo"]]}')
            (out / "systems.mmd").write_text("\n".join(sysmap))

# 4. Calculation lineage: sources -> calculations -> stores, grouped per
repo
ids: dict[str, str] = {}

def nid(key: str) -> str:
    return ids.setdefault(key, f"n{len(ids)}")

calc = ["flowchart LR"]
for r in reports:
    calc.append(f'    subgraph {rid[r["repo"]]}["'
{_label(r["repo"].split("/")[-1])}"]')
        for c in r["calculations"]:
            calc.append(f'        {nid(c["id"])}["{_label(c["output"]
["name"])}' =
{_label(c["formula"])}"]')
            calc.append("    end")
        for r in reports:

```

```

        for c in r["calculations"]:
            for i in c["inputs"]:
                if i["origin"] == "ingested" and i.get("source"):
                    calc.append(f'    {nid(_store_key(i["source"]))}[(("{
{_label(i["source"])}")}] '
                                f'-->|{_label(i["name"])}| {nid(c["id"])}]')
            for f in c["feeds"]:
                calc.append(f'    {nid(c["id"])} --> {nid(f)}')
            for s in c["output"]["stored_in"]:
                calc.append(f'    {nid(c["id"])} --> {nid(_store_key(s))}[(("{
{_label(s)}")}]')
            (out / "calculations.mmd").write_text("\n".join(calc))

    print(f"[report] wrote {out}/report.json, report.md, systems.mmd,
calculations.mmd")

```

run.py

```

"""Step 8 - Run it.

    python run.py net_revenue --alias netRevenue --alias NET_REVENUE
"""
import argparse
import asyncio
import sys

from agents import Agents, load_sourcegraph_tools
from config import SG_ENDPOINT, SG_TOKEN, load_repo_catalog
from main_graph import build_main_graph

async def main(element: str, aliases: list[str], catalog_path: str, out_dir:
str, verify: bool):
    if not SG_ENDPOINT or not SG_TOKEN:
        sys.exit("Set SRC_ENDPOINT and SRC_ACCESS_TOKEN first.")

    tools = await load_sourcegraph_tools()
    print("[mcp] tools:", [t.name for t in tools])

    graph = build_main_graph(Agents(tools), load_repo_catalog(catalog_path),
out_dir)
    await graph.ainvoke({"element": element, "aliases": aliases, "verify":
verify,

```

```

        "by_repo": {}, "repo_reports": [], "handoffs": [],
        "conflicts": []})

if __name__ == "__main__":
    p = argparse.ArgumentParser(description="Trace a business element,
separated by repository")
    p.add_argument("element")
    p.add_argument("--alias", action="append", default=[])
    p.add_argument("--repos", default="repos.yaml")
    p.add_argument("--out", default=None)
    p.add_argument("--no-verify", action="store_true", help="skip re-reading
cited lines")
    a = p.parse_args()
    asyncio.run(main(a.element, [a.element, *a.alias], a.repos,
                    a.out or f"lineage_{a.element}", not a.no_verify))

```

Your first run. Run the tests in the next section first. Then start small, so a bad prompt costs cents rather than dollars:

```

LINEAGE_MAX_REPOS=2 LINEAGE_MAX_DEPTH=1 python run.py net_revenue --alias
netRevenue

```

Open `lineage_net_revenue/report.md` and click a few citations. Once they land on the right lines, raise the limits back to their defaults. To preview the `.mmd` files, paste them into any Mermaid viewer, or into a Markdown file inside a mermaid code fence.

Testing

Test in four layers, cheapest first. Each layer catches a different kind of failure, and you only spend LLM money from layer 3 on.

Layer	Command	Needs	Catches
1. Offline tests	<code>pytest -q</code>	Nothing	Wiring, segregation, citation, and linking bugs
2. Tool contract	<code>python check_tools.py net_revenue</code>	Sourcegraph token	Missing tools, wrong endpoint, bad auth, different arg names

Layer	Command	Needs	Catches
3. Live smoke test	<code>LINEAGE_MAX_REPOS=2 LINEAGE_MAX_DEPTH=1 python run.py ...</code>	Sourcegraph + LLM	Prompts that don't find anything, bad line numbers
4. Golden-set evaluation	<code>python evaluate.py report.json golden/x.yaml</code>	A hand-checked answer	Quality drift when you change prompts or models

Layer 1: offline tests (no LLM, no Sourcegraph)

The tests replace `run_agent` and `run_llm` with fakes that return fixed findings, and replace the Sourcegraph tools with a fake `commit_search` and `read_file`. Everything else is the real code, running end to end. Create an empty `tests/__init__.py`, then the four files below.

`tests/fakes.py`

```

"""Fake agents and tools so the whole graph runs offline, with no LLM and no
Sourcegraph."""
from models import (Boundary, CalculationDraft, ConnectorResult,
                    DiscoveryResult, Edge,
                    InputDraft, Loc, OutputDraft, RepoProfile, Symbol,
                    TraceResult)

A = "github.com/acme/data-pipeline"
B = "github.com/acme/finance-dashboard"
SHA = "a" * 40

FILES = {
    (A, "models/fct_orders.sql"): "\n".join(
        [f"{n}  -- filler" for n in range(1, 40)]
        + ["40  select", "41      order_id,", "42      gross_amount -
discount_amount - refund_amount as net_revenue",
          "43  from stg_orders", "44  join stg_refunds using (order_id)"]),
    (A, "models/stg_orders.sql"): "1  select gross_amount, discount_amount
from {{ source('kafka', 'orders_v2') }}",
    (B, "metrics/revenue.py"): "\n".join([f"{n}  # filler" for n in range(1,
88)]
                                         + ["88  net_revenue = (gross_amount
- discount_amount) / 1.2"]),

```

```

}

class FakeTool:
    def __init__(self, name, fn):
        self.name, self._fn, self.args = name, fn, {}

    async def ainvoke(self, args):
        return self._fn(args)

TOOLS = [
    FakeTool("sg_commit_search", lambda a: f"commit {SHA} Merge pull request #12"),
    FakeTool("sg_read_file", lambda a: FILES.get((a["repo"], a["path"]), "")),
]

class FakeAgents:
    discovery, repo_profiler, connector = "discovery", "profiler", "connector"
    tools = TOOLS

    def tracers_for(self, repo, revision):
        return {r: (r, repo) for r in ("upstream", "downstream", "storage")}

def loc(path, s, e=None):
    return Loc(path=path, start_line=s, end_line=e or s)

async def fake_run_agent(agent, text):
    if agent == "discovery":
        return DiscoveryResult(symbols=[Symbol(repo=A, path="models/fct_orders.sql", name="net_revenue"),
                                         Symbol(repo=B, path="metrics/revenue.py", name="net_revenue")])
    role, repo = agent
    if repo == A and role == "upstream":
        return TraceResult(
            edges=[Edge(source="gross_amount", target="net_revenue", relation="produces", access="write",

```

```

        formula="gross_amount - discount_amount -
refund_amount",
        loc=loc("models/fct_orders.sql", 42)]],
        new_symbols=[Symbol(repo=B, path="leak.py", name="x"),      #
must be dropped
        Symbol(repo=A, path="models/stg_orders.sql",
name="gross_amount")])
    if repo == A and role == "storage":
        return TraceResult(boundaries=[
            Boundary(direction="in", channel="topic", name="kafka:orders.v2",
                carries=["gross_amount"],
loc=loc("models/stg_orders.sql", 1)),
            Boundary(direction="out", channel="table",
name="analytics.fct_orders.net_revenue",
                loc=loc("models/fct_orders.sql", 40, 44))])
    if repo == B and role == "storage":
        return TraceResult(boundaries=[
            Boundary(direction="in", channel="table",
name="analytics.fct_orders.net_revenue",
                loc=loc("metrics/revenue.py", 10))])
    return TraceResult()

async def fake_run_llm(llm, system, text):
    if llm == "connector":
        return ConnectorResult(conflicts=["finance-dashboard drops refunds
and strips VAT"])
    if A in text:
        return RepoProfile(role="producer", summary="Computes net_revenue in
dbt.", calculations=[
            CalculationDraft(
                formula="gross_amount - discount_amount - refund_amount",
                loc=loc("models/fct_orders.sql", 42),
                inputs=[InputDraft(name="gross_amount", origin="ingested",
source="kafka:orders.v2",
                                loc=loc("models/stg_orders.sql", 1)),
                    InputDraft(name="refund_amount",
origin="unresolved")],
                output=OutputDraft(name="net_revenue", stored_in=
["table:analytics.fct_orders.net_revenue"],
                                loc=loc("models/fct_orders.sql", 40,
44))))))
    return RepoProfile(role="producer+consumer", summary="Recomputes

```

```

net_revenue.", calculations=[
    CalculationDraft(
        formula="(gross_amount - discount_amount) / 1.2",
        loc=loc("metrics/revenue.py", 88),
        inputs=[InputDraft(name="net_revenue", origin="ingested",
source="table:analytics.fct_orders.net_revenue",
                                loc=loc("metrics/revenue.py", 10))],
        output=OutputDraft(name="dashboard_net_revenue",
loc=loc("metrics/revenue.py", 88))),
    CalculationDraft(formula="bad", loc=loc("metrics/revenue.py", 0),
# invalid line -> dropped
                                output=OutputDraft(name="x")))]])

```

tests/test_citations.py

```

from citations import build_url, finalize_calculation, make_citation,
numbered_lines, supports

def test_url_is_pinned_to_commit():
    url = build_url("github.com/acme/x", "abc123", "models/a b.sql", 40, 44)
    assert url.endswith("/github.com/acme/x@abc123/-/blob/models/a%20b.sql?
L40-44")

def test_url_without_pin_uses_default_branch():
    url = build_url("github.com/acme/x", "HEAD", "a.sql", 1, 1)
    assert "/github.com/acme/x/-/blob/a.sql?L1-1" in url

def test_bad_line_numbers_are_rejected():
    assert make_citation("r", "HEAD", {"path": "a.py", "start_line": 0,
"end_line": 3}) is None
    assert make_citation("r", "HEAD", {"path": "a.py", "start_line": 9,
"end_line": 3}) is None
    assert make_citation("r", "HEAD", None) is None

def test_calculation_without_valid_citation_is_dropped():
    draft = {"formula": "a+b", "loc": {"path": "x.py", "start_line": -1,
"end_line": 2},
            "output": {"name": "c"}}

```

```

    assert finalize_calculation("github.com/acme/x", "HEAD", draft) is None

def test_numbered_and_plain_file_output_both_parse():
    assert numbered_lines("1  a\n2  b = c")[2] == "b = c"
    assert numbered_lines("a\nb = c")[2] == "b = c"

def test_supports_checks_the_cited_window():
    lines = {i: "" for i in range(1, 60)}
    lines[42] = "gross - disc as net_revenue"
    assert supports(lines, 42, 42, ["net_revenue"])
    assert supports(lines, 40, 40, ["net_revenue"])          # within slack
    assert not supports(lines, 10, 12, ["net_revenue"])      # wrong place

```

tests/test_linking.py

```

from linking import link_calculations, match_boundaries, norm, trace_chain

def calc(cid, out, inputs):
    return {"id": cid, "formula": "f", "citation": {}, "feeds": [],
            "inputs": [{"name": n, "origin": "computed", "computed_by": []}
            for n in inputs],
            "output": {"name": out, "stored_in": []}}

def test_aliases_normalise():
    assert norm("net_revenue") == norm("netRevenue") == norm("NET-REVENUE")

def test_exact_boundary_match_creates_handoff():
    reports = [
        {"repo": "A", "boundaries": [{"direction": "out", "channel": "table",
            "name": "x.t.c"}]},
        {"repo": "B", "boundaries": [{"direction": "in", "channel": "table",
            "name": "X.T.C"}]},
    ]
    assert match_boundaries(reports) == [
        {"from_repo": "A", "to_repo": "B", "via": "table:x.t.c", "carries":
        [], "match": "exact"}]

```

```
def test_links_up_and_down_within_and_across_repos():
    reports = [
        {"repo": "A", "calculations": [calc("a1", "gross", []), calc("a2",
"net_revenue", ["gross"])]},
        {"repo": "B", "calculations": [calc("b1", "margin",
["netRevenue"])]},
    ]
    linked = link_calculations(reports, [{"from_repo": "A", "to_repo": "B"}])
    assert trace_chain(linked, "b1", "up") == ["a2", "a1"]
    assert trace_chain(linked, "a1", "down") == ["a2", "b1"]

def test_no_cross_repo_link_without_a_handoff():
    reports = [
        {"repo": "A", "calculations": [calc("a1", "net_revenue", [])]},
        {"repo": "B", "calculations": [calc("b1", "margin",
["net_revenue"])]},
    ]
    linked = link_calculations(reports, [])
    assert linked[1]["calculations"][0]["inputs"][0]["computed_by"] == []
```

tests/test_graph_offline.py

```
"""End-to-end run of the real graphs with fake agents and tools."""
import asyncio
import json

import main_graph
import repo_graph
from tests import fakes

def run(tmp_path, monkeypatch, verify=True):
    for mod in (main_graph, repo_graph):
        monkeypatch.setattr(mod, "run_agent", fakes.fake_run_agent)
        monkeypatch.setattr(mod, "run_llm", fakes.fake_run_llm)
    catalog = {fakes.A: {"system": "Analytics warehouse", "owner": "@data-
eng"}}
    graph = main_graph.build_main_graph(fakes.FakeAgents(), catalog,
str(tmp_path))
    asyncio.run(graph.ainvoke({"element": "net_revenue", "aliases":
["net_revenue"], "verify": verify,
                             "by_repo": {}, "repo_reports": [], "handoffs":
```

```

[], "conflicts": []}))
    return json.loads((tmp_path / "report.json").read_text())

def test_results_are_segreated_by_repo(tmp_path, monkeypatch):
    report = run(tmp_path, monkeypatch)
    repos = [r["repo"] for r in report["repositories"]]
    assert repos == sorted([fakes.A, fakes.B])
    for r in report["repositories"]:
        for c in r["calculations"]:
            assert c["citation"]["repo"] == r["repo"]

def test_calculation_has_pinned_citation_inputs_and_output(tmp_path,
monkeypatch):
    report = run(tmp_path, monkeypatch)
    a = next(r for r in report["repositories"] if r["repo"] == fakes.A)
    (c,) = a["calculations"]
    assert c["citation"]["commit"] == fakes.SHA
    assert f"@{fakes.SHA}/-/blob/models/fct_orders.sql?L42-42" in
c["citation"]["url"]
    assert c["citation"]["verified"] is True
    gross = next(i for i in c["inputs"] if i["name"] == "gross_amount")
    assert gross["origin"] == "ingested" and gross["source"] ==
"kafka:orders.v2"
    assert gross["citation"]["verified"] is True
    assert c["output"]["stored_in"] ==
["table:analytics.fct_orders.net_revenue"]

def test_invalid_citations_are_dropped(tmp_path, monkeypatch):
    report = run(tmp_path, monkeypatch)
    b = next(r for r in report["repositories"] if r["repo"] == fakes.B)
    assert [c["formula"] for c in b["calculations"]] == ["(gross_amount -
discount_amount) / 1.2"]

def test_repos_are_linked_by_handoff_and_calculations(tmp_path, monkeypatch):
    report = run(tmp_path, monkeypatch)
    assert {(h["from_repo"], h["to_repo"]) for h in report["handoffs"]} ==
{(fakes.A, fakes.B)}
    b = next(r for r in report["repositories"] if r["repo"] == fakes.B)
    assert b["calculations"][0]["inputs"][0]["computed_by"] == [

```

```
"calc:data-pipeline:models/fct_orders.sql:42"]
assert report["conflicts"]
```

```
def test_all_output_files_are_written(tmp_path, monkeypatch):
    run(tmp_path, monkeypatch, verify=False)
    for name in ("report.json", "report.md", "systems.mmd",
"calculations.mmd"):
        assert (tmp_path / name).stat().st_size > 0
```

Run them. `ANTHROPIC_API_KEY` only needs to be set, not valid, because nothing calls the model:

```
ANTHROPIC_API_KEY=dummy pytest -q
# ..... [100%]
# 15 passed
```

Layer 2: tool contract check

This confirms that your Sourcegraph endpoint exposes the tools the agents rely on, shows their argument names, and runs one real `keyword_search`. Run it whenever you change endpoints or Sourcegraph upgrades.

`check_tools.py`

```
"""Test 2 - Contract check: are the Sourcegraph MCP tools we rely on there,
and do they answer?

    python check_tools.py                # list tools and their arguments
    python check_tools.py net_revenue    # also run one real keyword_search
"""
import asyncio
import json
import sys

from agents import load_sourcegraph_tools, tool_named
from citations import as_text

REQUIRED = ["keyword_search", "read_file", "go_to_definition",
"find_references"]
OPTIONAL = ["commit_search"]  # used to pin citations to a commit; falls
back to HEAD
```

```

async def main(query: str | None):
    tools = await load_sourcegraph_tools()
    print("Tools on this server:", sorted(t.name for t in tools), "\n")

    missing = []
    for name in REQUIRED + OPTIONAL:
        t = tool_named(tools, name)
        status = "ok" if t else ("MISSING" if name in REQUIRED else "missing
(optional)")
        print(f"{name:18} {status}")
        if t:
            print("  args:", json.dumps(t.args)[:300])
        elif name in REQUIRED:
            missing.append(name)

    if query:
        kw = tool_named(tools, "keyword_search")
        key = "query" if "query" in kw.args else next(iter(kw.args))
        out = as_text(await kw.ainvoke({key: query}))
        print(f"\nkeyword_search({key}={query!r}) returned {len(out)}
chars:\n{out[:800]}")

    if missing:
        sys.exit(f"\nMissing required tools: {missing}. Use the /.api/mcp/all
endpoint.")
    print("\nContract OK")

if __name__ == "__main__":
    asyncio.run(main(sys.argv[1] if len(sys.argv) > 1 else None))

```

Check two things in its output. The argument names for `read_file` should include `repo` and `path`, plus `revision` if you pin commits. The `commit_search` output should contain a 40-character sha. If a name differs on your instance, adjust the dicts passed to `ainvoke` in `citations.py`.

Layer 3: live smoke test

Run one element on two repos, one wave deep, then read the log and the report.

```
LINEAGE_MAX_REPOS=2 LINEAGE_MAX_DEPTH=1 python run.py net_revenue --alias netRevenue
```

- ☐ `[discover]` lists the repos you expect
- ☐ Each repo prints at least one wave with edges greater than 0
- ☐ `citations verified: N/M` has N close to M
- ☐ Clicking a citation in `report.md` lands on the calculation
- ☐ No calculation in one repo's section cites a file from another repo

Layer 4: golden-set evaluation

Pick 3 to 5 elements you know well, and write down the right answer once by hand. After any change to prompts, models, or limits, re-run and score it. That's how you know a change helped.

golden/net_revenue.example.yaml

```
element: net_revenue
repos:
  github.com/acme/data-pipeline:
    role: producer
    calculations:
      - path: models/fct_orders.sql
        line: 42
  github.com/acme/finance-dashboard:
    role: producer+consumer
    calculations:
      - path: metrics/revenue.py
        line: 88
  github.com/acme/billing-service:
    role: source of inputs
handoffs:
  - {from: github.com/acme/billing-service, to: github.com/acme/data-pipeline}
  - {from: github.com/acme/data-pipeline, to: github.com/acme/finance-dashboard}
```

evaluate.py

```

"""Test 4 - Score a run against a hand-checked golden file.

    python evaluate.py lineage_net_revenue/report.json
golden/net_revenue.yaml --min-recall 0.8
"""

import argparse
import json
import sys

import yaml

LINE_SLACK = 3

def calc_matches(expected: dict, calc: dict) -> bool:
    c = calc["citation"]
    return (c["path"] == expected["path"]
            and c["start_line"] - LINE_SLACK <= expected["line"] <=
c["end_line"] + LINE_SLACK)

def evaluate(report: dict, golden: dict) -> dict:
    by_repo = {r["repo"]: r for r in report["repositories"]}
    exp_calcs = found = role_ok = 0
    got_calcs = sum(len(r["calculations"]) for r in report["repositories"])
    matched_got = 0
    for repo, exp in golden["repos"].items():
        got = by_repo.get(repo)
        if got and got["role"] == exp.get("role"):
            role_ok += 1
        for e in exp.get("calculations", []):
            exp_calcs += 1
            if got and any(calc_matches(e, c) for c in got["calculations"]):
                found += 1
        if got:
            matched_got += sum(1 for c in got["calculations"]
                               if any(calc_matches(e, c) for e in
exp.get("calculations", [])))

    exp_h = {(h["from"], h["to"]) for h in golden.get("handoffs", [])}
    got_h = {(h["from_repo"], h["to_repo"]) for h in report["handoffs"]}
    cites = [c["citation"] for r in report["repositories"] for c in
r["calculations"]]

```

```

    leaks = [c["id"] for r in report["repositories"] for c in
r["calculations"]
                if c["citation"]["repo"] != r["repo"]]
    return {
        "calc_recall": found / exp_calcs if exp_calcs else 1.0,
        "calc_precision": matched_got / got_calcs if got_calcs else 1.0,
        "role_accuracy": role_ok / len(golden["repos"]) if golden["repos"]
else 1.0,
        "handoff_recall": len(exp_h & got_h) / len(exp_h) if exp_h else 1.0,
        "citations_verified": sum(1 for c in cites if c.get("verified")) /
len(cites) if cites else 0.0,
        "repo_leaks": leaks,
    }

if __name__ == "__main__":
    p = argparse.ArgumentParser()
    p.add_argument("report")
    p.add_argument("golden")
    p.add_argument("--min-recall", type=float, default=0.8)
    a = p.parse_args()
    scores = evaluate(json.load(open(a.report)),
yaml.safe_load(open(a.golden)))
    for k, v in scores.items():
        print(f"{k:20} {v:.2f}" if isinstance(v, float) else f"{k:20} {v}")
    if scores["calc_recall"] < a.min_recall or scores["repo_leaks"]:
        sys.exit("FAIL")
    print("PASS")

```

Metric	Means	Aim for
calc_recall	Share of known calculations found, within 3 lines	0.8 or more
calc_precision	Share of reported calculations that are real	0.8 or more
role_accuracy	Repos given the right role	1.0
handoff_recall	Known system-to-system handoffs found	0.8 or more
citations_verified	Calculation citations confirmed by re-reading the file	0.9 or more
repo_leaks	Calculations filed under the wrong repo	Always empty

Sample input and output

This is what the offline test fixtures produce when run through the real graph and report writer. A live run has the same shape, with your repos, real commit shas, and more calculations.

Input

```
python run.py net_revenue --alias netRevenue
```

Console log

```
[discover] found the element in 2 repos: ['github.com/acme/data-pipeline',
'github.com/acme/finance-dashboard']
[repo] tracing github.com/acme/data-pipeline @ aaaaaaaaaaaaaa
[repo] tracing github.com/acme/finance-dashboard @ aaaaaaaaaaaaaa
  [github.com/acme/data-pipeline] wave 1: 1 edges, 2 boundaries, 1 queued
  [github.com/acme/finance-dashboard] wave 1: 0 edges, 1 boundaries, 0 queued
  [github.com/acme/data-pipeline] wave 2: 1 edges, 2 boundaries, 0 queued
  [github.com/acme/finance-dashboard] citations verified: 1/1 calculations
  [github.com/acme/data-pipeline] citations verified: 1/1 calculations
[report] wrote lineage_net_revenue/report.json, report.md, systems.mmd,
calculations.mmd
```

report.md (excerpt, commit shortened)

```
## github.com/acme/data-pipeline
```

```
System: Analytics warehouse | Owner: @data-eng | Role: producer | Revision:
`aaaaaaaaaaaa`
```

```
### `net_revenue = gross_amount - discount_amount - refund_amount`
```

```
Calculated at [models/fct_orders.sql:42-42]
```

```
(https://sourcegraph.acme.com/github.com/acme/data-
pipeline@aaaa.../-/blob/models/fct_orders.sql?L42-42) (verified)
```

```
Inputs:
```

```
- `gross_amount`: ingested from `kafka:orders.v2` - [models/stg_orders.sql:1-
1](...) (verified)
```

```
- `refund_amount`: unresolved - no citation
```

```
Output: `net_revenue` stored in `table:analytics.fct_orders.net_revenue` -
[models/fct_orders.sql:40-44](...) (verified)
```

```
Feeds: calc:finance-dashboard:metrics/revenue.py:88
```

```
## github.com/acme/finance-dashboard
```

```
System: unknown - add to repos.yaml | Owner: unknown | Role:
producer+consumer
```

```
### `dashboard_net_revenue = (gross_amount - discount_amount) / 1.2`
```

```
Calculated at [metrics/revenue.py:88-88](...) (verified)
```

```
Inputs:
```

```
- `net_revenue`: ingested from `table:analytics.fct_orders.net_revenue` <-
calc:data-pipeline:models/fct_orders.sql:42 - [metrics/revenue.py:10-10](...)
(NOT verified)
```

```
## Handoffs between systems
```

```
- github.com/acme/data-pipeline -> github.com/acme/finance-dashboard via
`table:analytics.fct_orders.net_revenue` (exact)
```

```
## Conflicts
```

```
- finance-dashboard drops refunds and strips VAT
```

calculations.mmd, rendered: sources flow into calculations, grouped by repository, and on into stores.

```
flowchart LR
  subgraph r0["data-pipeline"]
    n0["net_revenue = gross_amount - discount_amount - refund_amount"]
  end
  subgraph r1["finance-dashboard"]
    n1["dashboard_net_revenue = (gross_amount - discount_amount) / 1.2"]
  end
  n2[("kafka:orders.v2")] -->|gross_amount| n0
  n0 --> n1
  n0 --> n3[("table:analytics.fct_orders.net_revenue")]
  n3[("table:analytics.fct_orders.net_revenue")] -->|net_revenue| n1
```

How to read it:

- `refund_amount` shows as `unresolved` . That's a real gap to chase: the storage tracer didn't find where refunds come from.
- The `(NOT verified)` tag on `metrics/revenue.py:10` means the re-read didn't find `net_revenue` near line 10, so check that citation by hand.
- `finance-dashboard` shows `unknown` as owner because it's missing from `repos.yaml` .

Troubleshooting and tuning

Most problems show up in the smoke test log or in the evaluation scores. Find the symptom, then apply the fix.

Symptom	Likely cause	Fix
<code>check_tools.py</code> reports a tool <code>MISSING</code>	Core endpoint, or MCP disabled	Use <code>/.api/mcp/all</code> ; ask an admin to confirm <code>mcp.enabled</code>
<code>[discover]</code> found the element in <code>0 repos</code>	Literal search missed your naming	Pass more <code>--alias</code> spellings; add your idioms to <code>SEARCH_RECIPES</code> ; confirm the token can see those repos
Waves show <code>0 edges</code>	Tracer ran out of tool calls, or searched too broadly	Raise <code>LINEAGE_TOOL_CALL_LIMIT</code> ; add <code>lang:</code> and <code>-file:test</code> recipes

Symptom	Likely cause	Fix
Many citations NOT verified	Line numbers guessed, not read	Check that <code>read_file</code> output is numbered (Layer 2); keep <code>GROUNDING</code> in every prompt; lower <code>LINEAGE_MAX_SYMBOLS</code>
Citations all unchecked	<code>read_file</code> argument names differ	Match the names printed by <code>check_tools.py</code> in <code>verify_calculation</code>
Revision always HEAD	No <code>commit_search</code> , or no sha in its output	Set <code>LINEAGE_REVISION</code> to a tag or sha
Expected handoff missing	Boundary names not fully qualified the same way	Tighten the naming rule in <code>STORAGE_PROMPT</code> ; the connector catches near-misses
Input stays unresolved	Its source is outside the traced repos	Add that repo to the run, or accept the <code>in</code> boundary as its origin
<code>repo_leaks</code> not empty	A code change broke the segregation guard	Re-run <code>pytest</code> ; <code>test_results_are_segregated_by_repo</code> should fail and point to it
429 or rate-limit errors	Too many parallel calls	Lower <code>LINEAGE_CONCURRENCY</code> ; the retry middleware handles bursts
<code>GraphRecursionError</code>	An agent looped past its step limit	Raise <code>recursion_limit</code> in <code>run_agent</code> , or lower <code>LINEAGE_TOOL_CALL_LIMIT</code> so it stops earlier
Run is slow or expensive	Limits too high for the first pass	Start at <code>LINEAGE_MAX_REPOS=2</code> <code>LINEAGE_MAX_DEPTH=1</code> , then raise

Where to improve next:

- Add more golden elements. Every prompt change should move `calc_recall` up without dropping `calc_precision`.
- Save each `report.json` by date, so you can see when a calculation changed between releases.

- Load `report.json` into a graph database or data catalog if other teams need to query lineage.